Control PID del sistema Towercopter

Jesús Eduardo Gutiérrez Arias.

Juan Sebastián pardo Rojas

Facultad de ingeniería. Ingeniería electrónica

Universidad del Magdalena.

Abstracto Este informe presenta el diseño e implementación de un sistema de control para un monocóptero vertical de un grado de libertad. Se emplearon tres enfoques de control: PID, lógica difusa y redes neuronales, todos desarrollados en Matlab/Simulink y aplicados al mismo modelo físico. El sistema físico incluye un motor brushless, sensores de altura y una estructura guiada. Se describen el diseño, simulación, ajuste y evaluación comparativa de los controladores, destacando su desempeño en términos de estabilidad, precisión y respuesta dinámica.

1. INTRODUCCIÓN

En el campo del control automático, los sistemas con un grado de libertad representan una plataforma ideal para aplicar y comparar distintas estrategias de regulación. Este informe presenta el desarrollo de un sistema de control para un monocóptero vertical (towercopter), cuyo objetivo es mantener y seguir una altura deseada a través del ajuste de la velocidad de un motor brushless. Inicialmente, se implementó un controlador PID, dada su amplia aceptación y uso en la industria por su simplicidad y efectividad. Con el fin de evaluar alternativas más avanzadas, se desarrollaron también un controlador basado en lógica difusa y otro con redes neuronales artificiales, todos aplicados sobre el mismo modelo físico y de simulación del sistema.

El análisis comparativo de estos métodos de control busca identificar fortalezas, debilidades y el comportamiento dinámico de cada enfoque frente a variaciones en las condiciones de operación, perturbaciones y cambios en la referencia. Este trabajo tiene un enfoque práctico y formativo, integrando conocimientos teóricos con la implementación real en un entorno de laboratorio. Asimismo, permite al estudiante explorar el diseño, simulación, y validación de estrategias clásicas y modernas de control dentro de un mismo entorno experimental.

2. PALABRAS CLAVE

- Control PID
- Lógica difusa
- Redes neuronales
- Monocóptero
- Control automático
- Modelado
- Sistema dinámico

3. OBJETIVOS

3.1 OBJETIVO GENERAL

Diseñar, implementar y comparar distintas estrategias de control —PID, lógica difusa y redes neuronales— aplicadas a un monocóptero de un grado de libertad, con el fin de regular su altura mediante un motor brushless y evaluar el desempeño de cada enfoque.

3.2 OBJETIVOS ESPECÍFICOS

- Construir y documentar una estructura física que permita el movimiento vertical del motor en condiciones estables y seguras.
- Diseñar un controlador PID y ajustar sus parámetros utilizando métodos clásicos.
- Implementar un controlador difuso basado en funciones de membresía y una matriz de inferencia.
- Desarrollar una red neuronal para control del sistema, entrenada con datos obtenidos del comportamiento real o simulado.
- Evaluar el desempeño de cada estrategia mediante análisis cualitativo y cuantitativo.

4. MARCO TEÓRICO

El control de sistemas dinámicos es una disciplina fundamental en la ingeniería, permitiendo la regulación de variables como posición, velocidad, temperatura o presión mediante la aplicación de señales de control. En este contexto, el controlador PID (Proporcional-Integral-Derivativo) es uno de los más utilizados debido a su simplicidad, facilidad de implementación y buenos resultados en una amplia variedad de procesos industriales.

El controlador PID combina tres acciones básicas:

- **Proporcional (P):** responde en proporción al error actual, proporcionando una corrección inmediata.
- **Integral (I):** acumula el error a lo largo del tiempo, eliminando el error en estado estacionario.
- **Derivativo (D):** predice el comportamiento futuro del error, mejorando la respuesta transitoria del sistema.

La ecuación general del PID continuo es:

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt}$$

donde $u(t)$ es la señal de control, $e(t)$ es el error entre la señal de referencia y la salida del sistema, y K_p, K_i, y K_d son las ganancias proporcional, integral y derivativa respectivamente.

En este proyecto, el controlador PID fue implementado mediante la herramienta **PID Tuner** de Matlab, que permite estimar automáticamente un modelo del sistema a partir de datos y ajustar los parámetros del controlador para lograr un comportamiento deseado (como respuesta rápida, amortiguada o con prioridad a la robustez).

Más adelante, se explorarán otras dos estrategias de control: la lógica difusa, que se basa en reglas heurísticas y conjuntos difusos para tomar decisiones en presencia de incertidumbre, y las redes neuronales, que ofrecen un enfoque de aprendizaje basado en datos para modelar y controlar sistemas complejos no lineales.

5. DESCRIPCIÓN DEL SISTEMA Y MONTAJE FÍSICO

4.1 DESCRIPCIÓN GENERAL DEL SISTEMA Y SU FINALIDAD

El sistema desarrollado en este proyecto consiste en un monocóptero de un grado de libertad, conocido también como *towercopter*, cuyo propósito es controlar el movimiento vertical de un motor brushless con hélice montado en una plataforma guiada. El objetivo principal del sistema es alcanzar y mantener una altura específica definida por el usuario, mediante la regulación de la velocidad del motor.

La estructura física del proyecto está compuesta por una base rígida de madera, sobre la cual se disponen dos guías metálicas paralelas en posición central que permiten el desplazamiento vertical del motor. Este motor está soportado mediante una guía impresa en 3D, diseñada para encajar con precisión en las barras metálicas laterales. Más adelante se ilustra esta composición.

Esta estructura proporciona estabilidad y alineación al movimiento, lo que facilita la observación del comportamiento del sistema bajo diferentes condiciones de control. La hélice, montada en la parte superior del motor, genera la fuerza de empuje necesaria para elevar la plataforma, contrarrestando tanto el peso como la fricción del sistema.

El sistema integra un microcontrolador ESP32, encargado del procesamiento de señales y la comunicación con el entorno de simulación a través del **Arduino IDE**, donde se carga el código que contiene los parámetros del controlador PID. En este entorno, el usuario también puede ingresar los valores de altura deseados mediante el monitor serial.

El control del sistema se implementa en lazo cerrado, utilizando diferentes enfoques de control automático (PID, lógica difusa y redes neuronales), con el objetivo de mantener la altura deseada frente a perturbaciones externas, fricción, retardos de respuesta y la dinámica no lineal del sistema.

6. MONTAJE Y DOCUMENTACIÓN DEL MONTAJE FÍSICO

6.1. ESQUEMA DE MONTAJE Y COMPONENTES UTILIZADOS

El montaje físico del proyecto se llevó a cabo sobre una estructura tipo torre, construida en madera, con dimensiones aproximadas de 1 metro de altura, 50 cm de ancho y 5 cm de separación entre las guías laterales. Esta estructura permite un desplazamiento vertical estable del motor y garantiza la rigidez mecánica necesaria durante la operación.

A continuación, se presenta una imagen de referencia de la estructura diseñada.

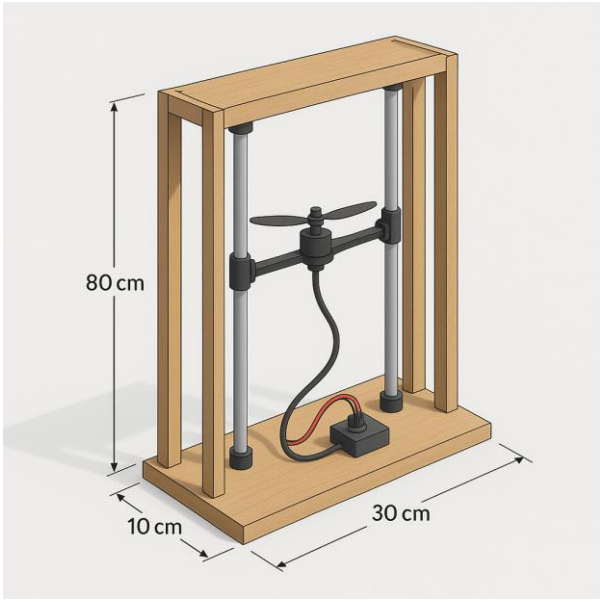


Figura 1. *Diseño preliminar de la estructura física del monocóptero con un grado de libertad [Fuente: elaboración propia]*

A continuación, se listan los principales componentes utilizados:

- **Motor brushless EMAX XA2212 (1400KV):** responsable de generar el empuje vertical mediante una hélice.
- **Hélice de 16 pulgadas:** acoplada al eje del motor, proporciona la fuerza de elevación.
- **ESC (Electronic Speed Controller) EMAX Simon Series 40A:** convierte la señal PWM del Arduino en corriente adecuada para accionar el motor.
- **Microcontrolador ESP32**: encargado de la lectura de sensores y del procesamiento de datos mediante el código cargado en el entorno Arduino IDE, donde se definen los valores del controlador PID y se establece la comunicación serial con el usuario para el ingreso de parámetros como la altura deseada.
- **Sensor de distancia ultrasónico:** montado en la parte superior de la estructura, mide la distancia del motor que se encuentra abajo hasta la parte superior donde está el sensor. Utiliza pulsos ultrasónicos, permitiendo determinar la altura desde arriba hacia abajo.
- **Guías laterales metálicas:** aseguran el movimiento vertical controlado del motor sin desviaciones laterales.
- **Fuente de alimentación de 12 V 10A:** energiza tanto el ESC como el Arduino y demás módulos.

El sistema de control se comunica con el Arduino por medio del puerto serial, enviando comandos de velocidad en función de la señal de control generada por los diferentes controladores. El sensor de altura entrega retroalimentación continua para cerrar el lazo de control.

6.2. RETOS DE CONSTRUCCIÓN Y SOLUCIONES IMPLEMENTADAS

Durante la fase de construcción del monocóptero se enfrentaron diversos desafíos técnicos, tanto mecánicos como electrónicos. A continuación, se describen los principales problemas identificados y las soluciones aplicadas para garantizar el correcto funcionamiento del sistema:

1. Estabilidad mecánica y guiado vertical

- **Problema:** En las primeras pruebas, el motor presentaba movimientos irregulares y oscilaciones laterales debido a una alineación imperfecta de las guías.
- **Solución:** Se reforzó la estructura base para mejorar su rigidez y estabilidad general. Además, se reajustaron los puntos de contacto de la plataforma móvil y se rediseñó la guía de soporte del motor, tomando en cuenta las dimensiones precisas del diámetro de las barras guía. Esto permitió minimizar la fricción y el rozamiento excesivo, garantizando un desplazamiento vertical suave, alineado y controlado a lo largo de toda la estructura.

2. Interferencias en la medición de altura

- **Problema:** La fuerza del viento generada por la hélice durante el ascenso afectaba la estabilidad de las mediciones del sensor ultrasónico, el cual inicialmente estaba ubicado en la base de la estructura. Las turbulencias generadas por las aspas del motor provocaban lecturas inestables e imprecisas.
- **Solución:** Se reubicó el sensor ultrasónico en la parte superior de la estructura, apuntando hacia la parte superior del motor. Esta nueva ubicación evitó la interferencia directa de las corrientes de aire generadas por las hélices, lo que permitió obtener una referencia más clara y sin perturbaciones mecánicas.

3. Calibración del ESC

- **Problema:** El ESC no respondía correctamente a las señales PWM generadas por el Arduino, lo que impedía controlar con precisión la velocidad del motor.
- **Solución:** Se realizó una calibración manual del ESC definiendo los valores máximos y mínimos del ancho de pulso (PWM) y se ajustó el código en el Arduino para asegurar compatibilidad total con el controlador de velocidad.

Gracias a estas soluciones, se logró un sistema mecánicamente estable, con buena capacidad de medición y control, apto para la aplicación de distintas estrategias de control automático.

A continuación, se presenta el diseño final de la estructura física del monocóptero, ya corregida y optimizada tras superar los desafíos mencionados en la etapa de construcción. La estructura cuenta con una base rígida, guías alineadas con baja fricción, y un sistema de montaje balanceado para el motor y la hélice, lo que garantiza un desplazamiento vertical estable y seguro. Esta versión funcional ha sido validada para operar de forma confiable bajo control automático.



Figura 2. *Vista final de la estructura funcional del monocóptero tras ajustes mecánicos y corrección de fricción [Fuente: elaboración propia]*.

7. IMPLEMENTACION DEL SISTEMA

La implementación del sistema de control comienza con la obtención de datos experimentales necesarios para modelar el comportamiento dinámico del monocóptero. Esta etapa es fundamental, ya que proporciona la base sobre la cual se diseña y ajusta el controlador PID utilizando herramientas de identificación y simulación en Matlab.

7.1 ADQUISICIÓN DE DATOS

El proceso de adquisición de datos se realizó utilizando un microcontrolador **ESP32**, programado en el entorno **Arduino IDE**. La comunicación con el sistema se llevó a cabo mediante comandos enviados por el monitor serial. El usuario ingresa una velocidad en porcentaje (0–100%) y, tras un retardo programado de activación, el motor brushless es controlado mediante un **ESC (Electronic Speed Controller)** a través de señales PWM generadas por la ESP32.

Durante las pruebas, se determinó que el valor mínimo de PWM necesario para iniciar el despegue del monocóptero es de **45 %**, mientras que la altura máxima dentro de la estructura se alcanza con aproximadamente **2.21 segundos**. Estas señales se mapearon desde un rango de entrada de 0 a 100% para facilitar el control desde la interfaz serial.

El sensor ultrasónico conectado a la ESP32 se utilizó para medir la distancia entre la base del sistema y la plataforma móvil. Esta medición se realizó a una frecuencia de **20 ms por muestra**, frecuencia que fue verificada como adecuada tanto por la hoja de datos del sensor como por pruebas de medición en tiempo real. Este período de muestreo es compatible con los requisitos del sistema para control en lazo cerrado.

La señal de altura (en centímetros) fue registrada junto con el tiempo transcurrido desde el inicio del movimiento del motor hasta que la plataforma alcanzó su altura máxima. El arranque del motor se activaba mediante una señal enviada desde el monitor serial. Durante el ascenso del motor, los datos fueron capturados utilizando una herramienta complementaria en Excel, lo que permitió registrar y almacenar la información para su posterior análisis.

7.2 CÓDIGO DE ADQUISICIÓN DE DATOS

El código en Arduino fue implementado en la ESP32 para controlar el motor mediante el ESC y capturar datos de altura promedio en tiempo real.

El código implementado en la ESP32 permite controlar un motor mediante un ESC utilizando señales PWM, y medir en tiempo real la altura alcanzada con un sensor ultrasónico. Al recibir una velocidad desde el monitor serial, el sistema espera 15 segundos antes de activar el motor y comenzar a registrar datos.

El sensor ultrasónico realiza mediciones enviando pulsos sonoros a través del pin **TRIG** y midiendo el tiempo que tarda en recibir el eco por el pin **ECHO**, calculando así la distancia. En el código implementado, mientras el sistema está activo, se realiza una medición continua de la altura, almacenando hasta 20 valores válidos en un arreglo circular. Una vez el arreglo se llena, se calcula un **promedio** de esas 20 muestras, lo cual permite filtrar lecturas ruidosas o atípicas. Esta estrategia proporciona una estimación más precisa y estable de la altura real, la cual se transmite junto con el tiempo transcurrido desde la activación del sistema.

7.3 IMPORTACIÓN Y USO EN MATLAB

Una vez recolectados los datos experimentales, se importaron a Matlab para construir un objeto de identificación utilizando la función $iddata$, parte de la ***System Identification Toolbox***. Este objeto contiene los valores de altura medidos a intervalos regulares de **20 milisegundos**, totalizando **134 muestras** de altura y tiempo. Estas fueron la base para la estimación del modelo dinámico del sistema:

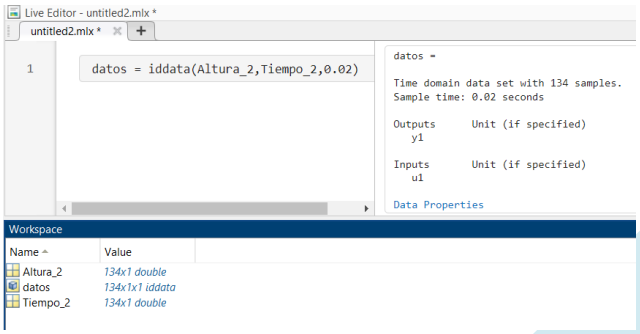


Figura 3. *Entorno inicial de trabajo en Matlab con los datos experimentales importados*
[Fuente: elaboración propia]

7.4 SINTONIZACIÓN DEL CONTROLADOR

Paso a paso del uso de *PID Tuner*

A continuación, se describe detalladamente el procedimiento seguido para configurar y utilizar la herramienta *PID Tuner* en *Matlab*:

1. **Selección del tipo de controlador**

En la barra superior de PID Tuner, se seleccionó el tipo de controlador **PID (Proporcional–Integral–Derivativo)**, que es el enfoque clásico adoptado para este proyecto. **(Ver Figura 4: Entorno del PID Tuner)**

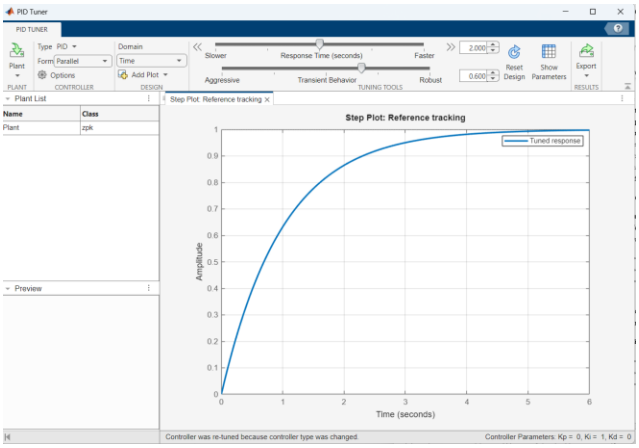


Imagen 4. *Entorno principal de la herramienta PID Tuner en Matlab*
[Fuente: elaboración propia]

2. **Identificación de nueva planta: "Identify New Plant"**

Se eligió la opción **"Identify New Plant"**, que permite modelar el comportamiento del sistema a partir de los datos experimentales previamente importados.
(Ver Figura 5: Identificación de la planta)

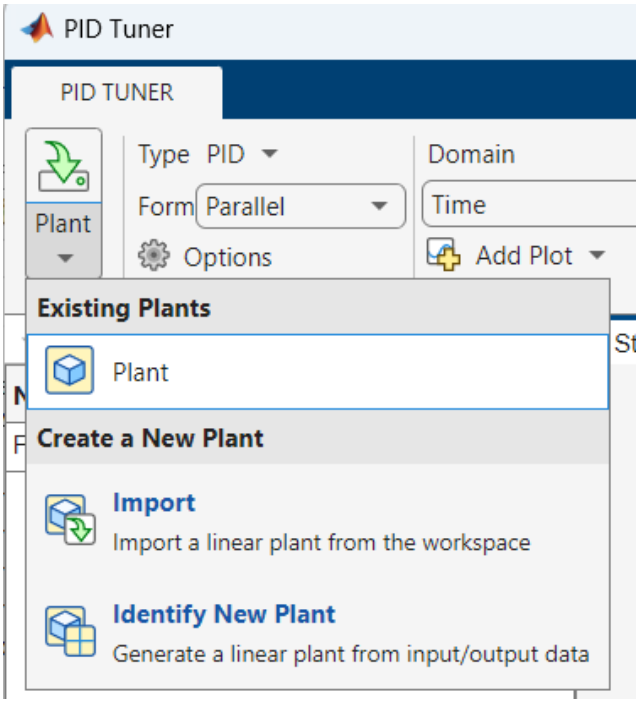


Imagen 5. *Identificación de la planta a partir de los datos experimentales en PID Tuner*
[Fuente: elaboración propia]

3. **Carga de datos de entrada/salida: "Get I/O Data"**

Se accedió a **"Get I/O Data"**, donde se seleccionó la opción **"Step Response: Import plant response from step (bump) test"**, ya que el sistema fue sometido a una entrada tipo escalón y se midió su salida. Esta configuración es ideal

para estimar ganancia, constantes de tiempo y posibles retardos.

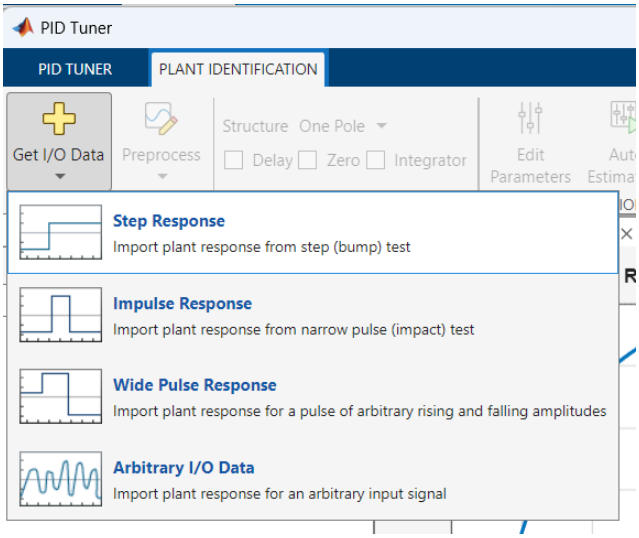


Imagen 6. *Carga de datos de entrada y salida del sistema mediante la opción Get I/O Data en PID Tuner*

4. Importación de los datos al PID Tuner

Se especificó como entrada el nombre de la variable que contenía el objeto *iddata*. Matlab utiliza esta información para generar un modelo inicial del sistema, tomando en cuenta:

- La señal de entrada.
- La salida medida.
- El tiempo de muestreo.

(Ver Figura 7: Importación de la señal de salida del sistema).

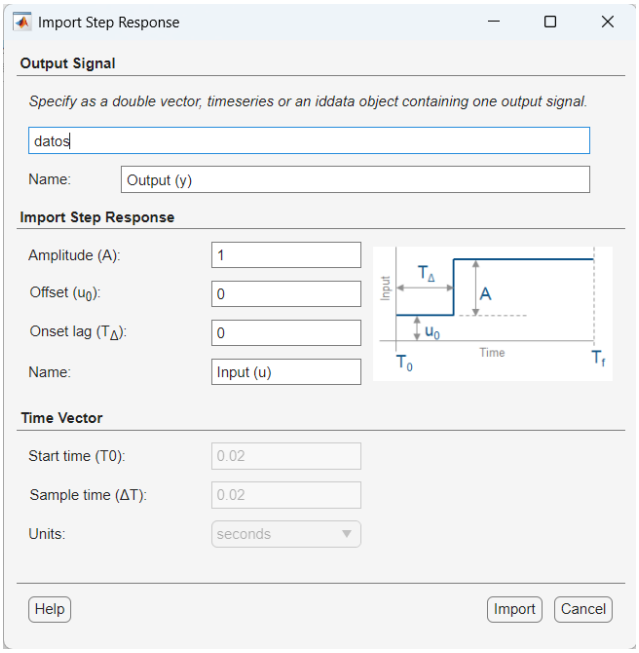


Imagen 7. *Importación de la señal de salida del sistema utilizando la función Step Response en PID Tuner.*

5. Selección de la estructura de la planta

PID Tuner ofrece varias configuraciones de modelo para representar la planta estimada:

- One Pole
- Two Real Poles
- Underdamped Pair
- Underdamped Pair + Real Pole

Se exploraron estas opciones y se seleccionó la que proporcionó el mejor ajuste según los indicadores de desempeño. **(Ver Figura 8: Tipos de modelos de planta)**

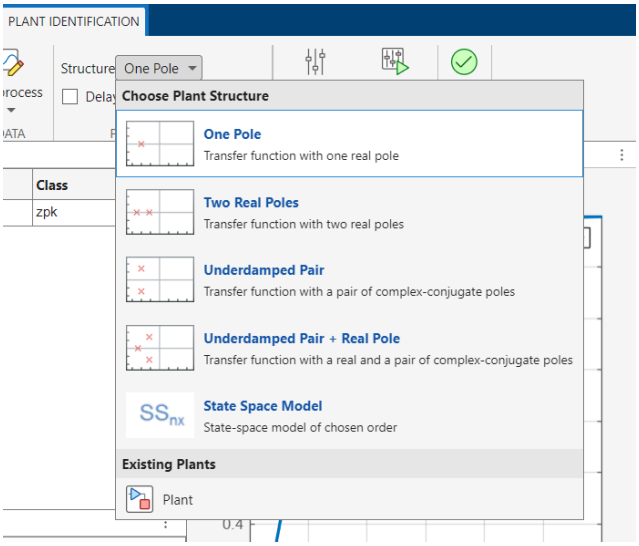


Imagen 8. *Selección del tipo de modelo de planta en PID Tuner*
[Fuente: elaboración propia]

6. Evaluación de las respuestas de las plantas estimadas

Para cada modelo, Matlab estimó los parámetros del controlador PID y generó un gráfico de respuesta al escalón. Se analizaron los siguientes indicadores:

- ***Kp*** : Ganancia proporcional.
- ***Ki*** : Ganancia integral.
- ***KD***: Ganancia derivativa.
- ***N***: Filtro derivativo.
- ***Fit to estimation data*** (%): Calidad del ajuste respecto a los datos reales.
- ***FPE*** (***Final Prediction Error***): Error medio esperado al predecir nuevos datos.
- ***PID Tuned Values*:** Corresponden a los parámetros del controlador proporcional–integral–derivativo (PID) calculados automáticamente por MATLAB con base en el modelo estimado. Estos valores determinan el comportamiento del sistema controlado, incluyendo su estabilidad, rapidez de respuesta y error en estado estacionario.
- ***Kp*** (***Ganancia proporcional***): Define la magnitud de la respuesta del controlador frente al error instantáneo. A mayor Kp, mayor corrección, pero puede inducir oscilaciones si es excesivo.
- ***Ki*** (***Ganancia integral***): Corrige el error acumulado en el tiempo. Reduce el error en estado estacionario, pero si es muy alta puede generar inestabilidad.
- ***Kd*** (***Ganancia derivativa***): Aporta una acción anticipativa al controlador al considerar la velocidad de cambio del error. Mejora la estabilidad y reduce el sobreimpulso.
- ***Fit to estimation data*** (%): Indica qué tan bien el modelo ajustado representa los datos reales del sistema. Un valor cercano al 100% sugiere un buen ajuste; un valor bajo indica que el modelo no reproduce con precisión el comportamiento del sistema.
- ***FPE*** (***Final Prediction Error***): Estima el error promedio que tendría el modelo al predecir nuevos datos. Un valor menor significa un modelo más preciso y confiable.

7. MODELOS DE PLANTA IDENTIFICADOS MEDIANTE PID TUNER

- One Pole
  - Fit to estimation data: 88.47%
  - FPE: 6.67559

Este modelo representa una planta de primer orden, ideal para sistemas con una única constante de tiempo dominante. Aunque el ajuste del 88.47% indica que la planta capta una parte considerable del comportamiento dinámico del sistema, el FPE alto (6.67559) refleja una capacidad limitada para predecir datos futuros con precisión. Esto sugiere que el sistema real posee una dinámica más compleja, probablemente influida por múltiples polos o efectos de oscilación que no pueden ser representados con una única constante de tiempo. A pesar de ser simple y de fácil implementación, este modelo fue descartado por no reflejar fielmente la respuesta del monocóptero.

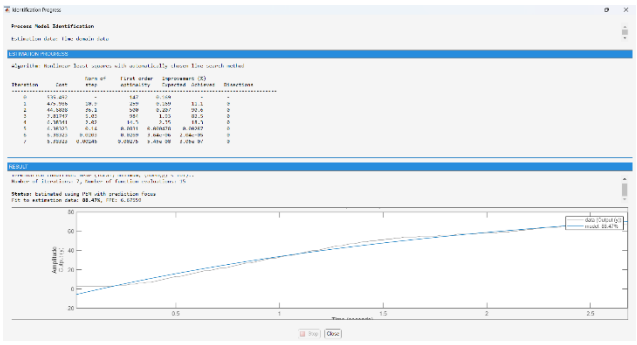


Imagen 9. *Resultado de la planta estimada con modelo One Pole en PID Tuner*
[Fuente: elaboración propia]

- Two Real Poles
  - Fit to estimation data: 95.64%
  - FPE: 0.986346

Este modelo incorpora dos polos reales, permitiendo capturar dos procesos dinámicos independientes en el sistema. Con un ajuste del 95.64%, igual al del modelo con polos complejos, y un FPE muy bajo (0.986346), este modelo demuestra una alta fidelidad en la representación del comportamiento del monocóptero. Al no incluir componentes oscilatorios, representa una opción más conservadora y estable, ideal para sistemas que no presentan sobreimpulsos o resonancias marcadas. Su simplicidad relativa frente al modelo oscilatorio puede ofrecer ventajas en términos de robustez y facilidad de implementación, aunque podría limitar su capacidad de modelar respuestas subamortiguadas.

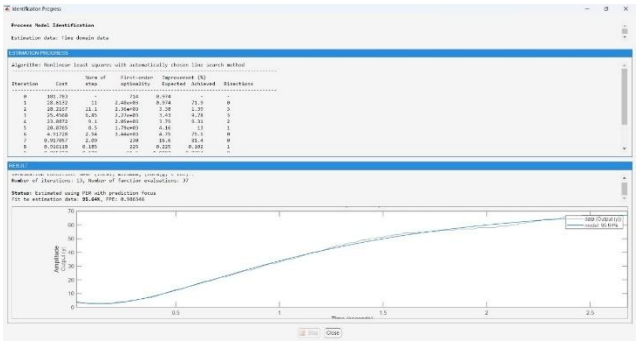


Imagen 10. *Resultado de la planta estimada con modelo Two Real Poles en PID Tuner*
[Fuente: elaboración propia]

- Underdamped Pair
  - Fit to estimation data: 92.02%
  - FPE: 3.29714

Este modelo utiliza un par de polos complejos conjugados, ideal para representar sistemas con comportamientos oscilatorios o subamortiguados. Sin embargo, a pesar de capturar esa dinámica natural de forma teórica, el ajuste a los datos reales es de 92.02%, y el FPE (3.29714) es considerablemente más alto en comparación con otros modelos evaluados. Esto sugiere que, aunque el sistema pueda presentar alguna oscilación, la estructura de polos complejos no fue la más adecuada para describir con precisión su comportamiento real. Por tanto, este modelo fue descartado frente a otros con mayor exactitud y menor error de predicción, como el de Two Real Poles.

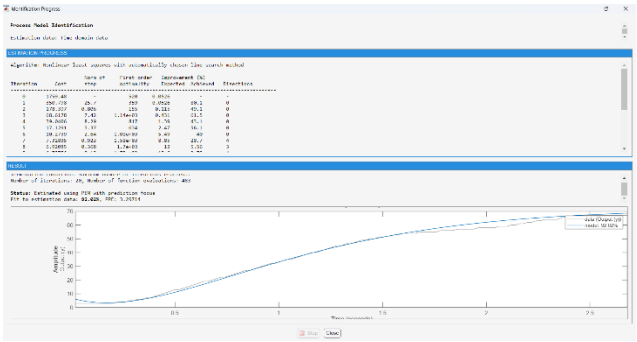


Imagen 11. *Resultado de la planta estimada con modelo Underdamped Pair en PID Tuner*
[Fuente: elaboración propia]

- Underdamped Pair + Real Pole
  - Fit to estimation data: 95.51%
  - FPE: 1.07638

Este modelo combina un par de polos complejos conjugados (que representan comportamiento oscilatorio) con un polo real adicional, lo que permite capturar tanto las oscilaciones rápidas como una dinámica más lenta o retardos del sistema. El ajuste a los datos reales es muy alto (95.51%) y el error de predicción relativamente bajo (FPE = 1.07638), lo que indica una muy buena capacidad de modelado.

Sin embargo, al compararlo con el modelo de Two Real Poles, que alcanza un ajuste ligeramente mayor (95.64%) y un FPE menor (0.986346), se concluye que el incremento en complejidad del modelo no se traduce en una mejora significativa en precisión. Por tanto, aunque es una opción válida y robusta, no fue seleccionada como la planta final para el diseño del controlador.

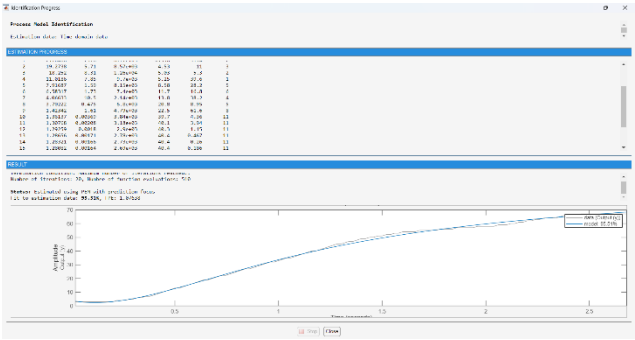


Imagen 12. *Resultado de la planta estimada con modelo Underdamped Pair + Real Pole en PID Tuner*
[Fuente: elaboración propia]

7.1 Selección del modelo de planta más adecuado

Luego de analizar los diferentes modelos propuestos por la herramienta PID Tuner, se seleccionó como planta definitiva el modelo de **Two Real Poles**. Esta decisión se basó en varios criterios clave:

- **Alta precisión en el ajuste a los datos reales**, con un **Fit to estimation data del 95.64%**, el más alto entre todas las opciones evaluadas.
- **Bajo error de predicción** con un **FPE de 0.986346**, lo que garantiza un modelo confiable para representar el comportamiento futuro del sistema.
- **Estabilidad estructural del modelo**, ya que al no involucrar polos complejos, se reduce la posibilidad de oscilaciones no deseadas, lo que lo convierte en una opción más robusta para sistemas físicos reales.
- **Menor complejidad comparativa**, en relación con modelos de tres polos o con pares subamortiguados, lo que facilita su análisis, simulación y eventual implementación en hardware.

Este modelo logra el equilibrio ideal entre precisión, simplicidad y estabilidad, por lo cual se consideró **el más eficiente** para representar la dinámica del monocóptero en el contexto de este proyecto.

8. EXPORTACIÓN DEL MODELO DE PLANTA IDENTIFICADO Y DEL CONTROLADOR PID

Una vez validada la respuesta del modelo estimado, se utilizó la opción **"Export"** para enviar la planta identificada y los parámetros del controlador PID al espacio de trabajo de Matlab. **(Ver Figura 9: Exportación de la planta)**

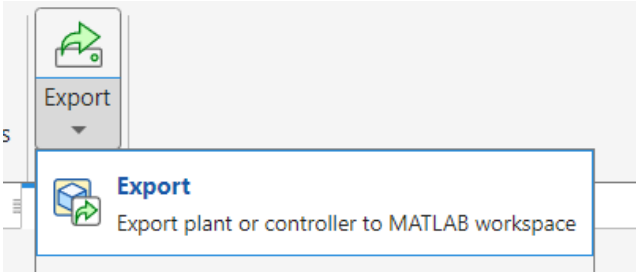


Imagen 14. *Exportación del modelo de planta identificado en PID Tuner*
[Fuente: elaboración propia]

Esta opción permite **exportar el modelo de planta identificado** o el **controlador PID ajustado** desde la interfaz gráfica del PID Tuner al espacio de trabajo de MATLAB. Al hacerlo, los datos quedan disponibles para su uso directo en scripts, simulaciones o análisis posteriores.

Se procede a seleccionar la planta con mejores resultados y ya se tendría en el espacio de trabajo de Matlab estos datos.

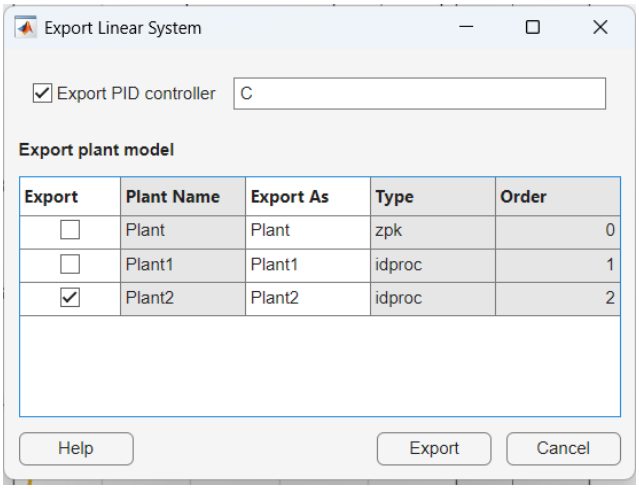


Imagen 15. *Selección del modelo de planta más adecuado en PID Tuner*

[Fuente: elaboración propia]

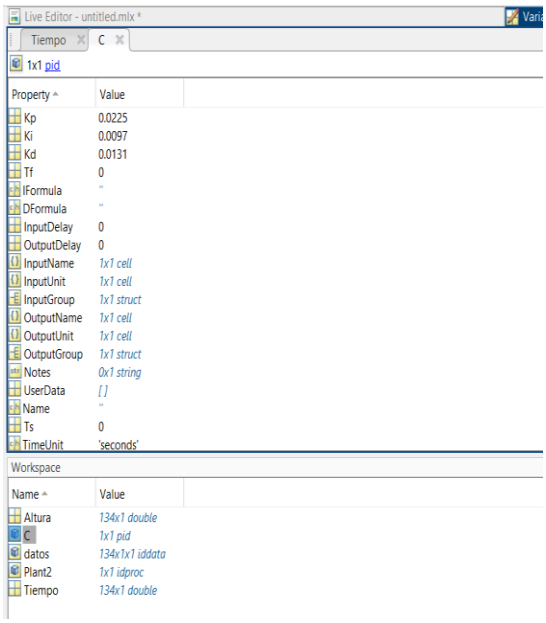


Imagen 16. *Parámetros y resultados obtenidos de la identificación de la planta en PID Tuner*

[Fuente: elaboración propia]

9. IMPLEMENTACIÓN DEL SISTEMA EN SIMULINK Y MEJORA DE LA RESPUESTA EN LAZO CERRADO

Con los datos exportados, se construyó un modelo de lazo cerrado en Simulink, donde el controlador PID regula la planta modelada como una función de transferencia de segundo orden. **(Ver Figura 17: Diagrama del sistema en Simulink)**

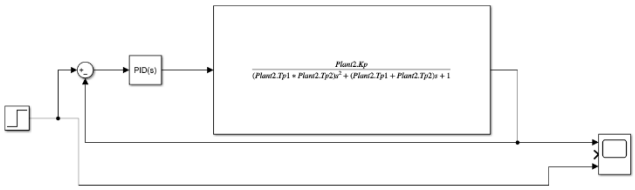


Figura 17. *Diagrama del sistema implementado en Simulink*

[Fuente: elaboración propia]

9.1. Objetivo de la simulación:

Analizar cómo el controlador PID regula la salida del sistema frente a una entrada escalón, observando:

- Tiempo de establecimiento
- Sobreimpulso
- Error en estado estacionario
- Oscilaciones

9.2 Análisis de la respuesta simulada

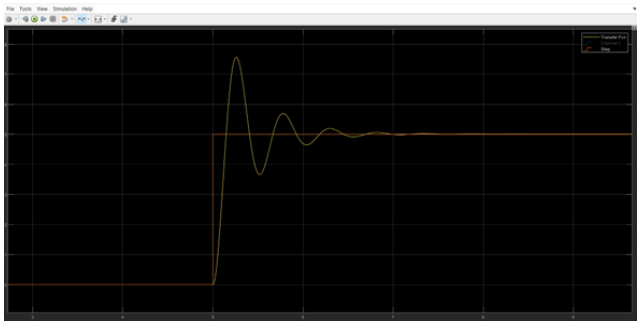


Figura 18. *Respuesta simulada en Simulink del sistema con controlador PID aplicado a la planta identificada Fuente: Matlab*

Análisis de la respuesta simulada y comparación con la práctica

Inicialmente, al observar la respuesta del sistema en Simulink, se evidencia un **sobreimpulso significativo** tras la aplicación de una entrada escalón. Sin embargo, la salida tiende a estabilizarse progresivamente, mostrando un comportamiento típico de un sistema subamortiguado que finalmente alcanza el estado estacionario.

No obstante, al implementar este modelo en el sistema físico real, se identifica una **diferencia importante en el tiempo de estabilización**, ya que el sistema tarda considerablemente más en alcanzar una respuesta estable e incluso no llega claramente a un punto de equilibrio. Esta discrepancia puede atribuirse a factores como:

- **No linealidades y retardos** presentes en el sistema real que no fueron modelados.
- **Ruido y perturbaciones externas**.
- **Limitaciones físicas** de los actuadores o sensores.

Para corregir esta diferencia y mejorar el desempeño del sistema, se recurrió al **ajuste manual de los parámetros del controlador PID**. A través de iteraciones sucesivas, se logró:

- **Reducir el tiempo de respuesta**, evitando que el sistema tarde demasiado en estabilizarse.
- **Disminuir la agresividad del controlador**, evitando sobreimpulsos y oscilaciones excesivas.
- **Aumentar la robustez del sistema**, haciéndolo menos sensible a perturbaciones y errores de modelado.

9.3 Valores obtenidos inicialmente

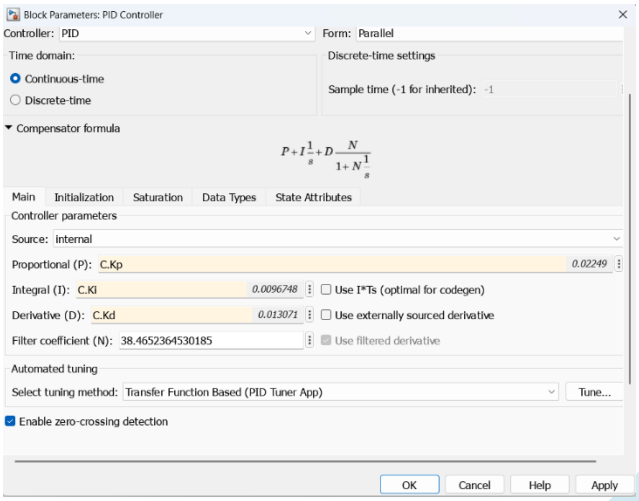


Figura 19. Valores del controlador PID obtenidos a partir del modelo de la planta.

Fuente: Simulación en MATLAB.

Parámetros del PID:

- $Kp = 0.02249$
- $Ki = 0.0096748$
- $Kd = 0..013071$

Conversión a forma natural del PID:

- $Kp = 2.43031999408004$
- $Ki = \frac{kp}{ki} \rightarrow 2.32$
- $Kd = kp * kd \rightarrow 0.000293$

Al implementar estos valores en el sistema físico, se evidenció un comportamiento inestable: el controlador generaba oscilaciones constantes sin alcanzar un punto claro de estabilidad. Esto indicó que el diseño inicial no era adecuado para las condiciones reales de operación.

Por tal motivo, se procedió a una optimización manual del controlador utilizando las herramientas de ajuste proporcionadas por MATLAB/Simulink, tal como se describió anteriormente. Este ajuste permitió mejorar tanto la estabilidad como el tiempo de respuesta del sistema.

9.4 Punto óptimo final

Luego de realizar múltiples ajustes y pruebas mediante la herramienta **PID Tuner** de MATLAB, se logró obtener un conjunto de parámetros óptimos para el controlador PID, con los cuales el sistema muestra un comportamiento estable, con rápida convergencia y una respuesta adecuada frente a una entrada tipo escalón.

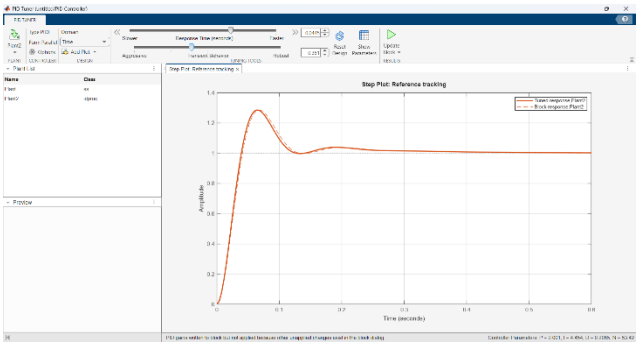


Imagen 20. *Herramienta de ajuste manual mediante PID Tuner.*

Fuente: Simulación en MATLAB.

Parámetros finales del PID (forma paralela):

- $Kp = 2.001$
- $Ki = 4.454$
- $Kd = 0.2065$

Conversión a forma natural del PID:

- $Kp = 2.001$

- $Ki = 0.44$
- $Kd = 0.41$

Estos valores fueron seleccionados a partir de una simulación donde se buscó un equilibrio entre **tiempo de respuesta rápido**, **comportamiento poco oscilatorio** y **robustez ante perturbaciones**. Como se muestra en la Figura 20, la respuesta presenta:

- Un sobreimpulso moderado, menor al 30%.
- Un tiempo de establecimiento inferior a 0.3 segundos.
- Error en estado estacionario prácticamente nulo.

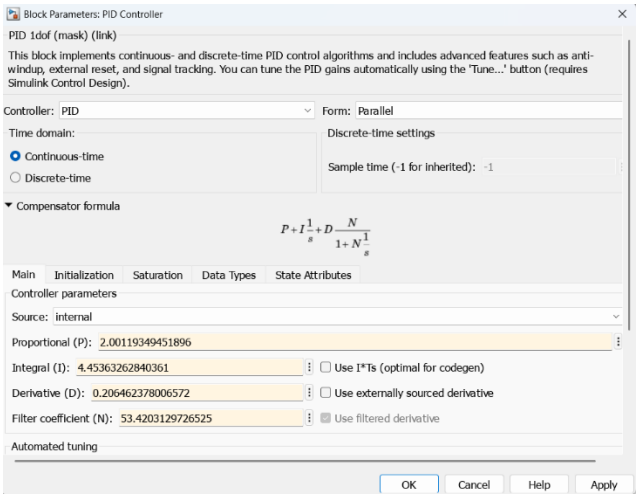


Imagen 21. *Datos finales del controlador PID tras ajuste mediante el bloque Landa Tuning.*

```
// Parámetros PID (ajustados para respuesta activa)
double Kp = 2.00119349451896;
double Ki = 0.44933959791;
double Kd = 0.41317116773;
```

Imagen 23. Implementación del controlador PID en forma natural en Arduino.

Fuente: Desarrollo propio.

2. Características observadas en la salida (línea amarilla)

- **Sobreimpulso (Overshoot):** La respuesta inicial supera visiblemente el valor final de referencia (6), alcanzando un pico cercano a **8.8**. Esto representa un sobreimpulso aproximado del **47%**, lo cual indica un sistema **subamortiguado**.
- **Estabilización:** Tras el sobreimpulso, la señal decae rápidamente y entra en una fase de oscilación que se amortigua progresivamente, hasta estabilizarse alrededor del valor deseado (**6 unidades**) en un tiempo relativamente corto (≈ 0.6 s).
- **Error en estado estacionario:** La señal alcanza de forma precisa el valor de referencia, por lo que se puede considerar que el **error en estado estacionario es prácticamente nulo**.
- **Tiempo de establecimiento (Settling Time):** El sistema parece estabilizarse completamente antes de **t = 6 s**, lo que da un **tiempo de establecimiento menor a 1 segundo** (desde t = 5 s). Esto es aceptable o incluso óptimo dependiendo de la aplicación.

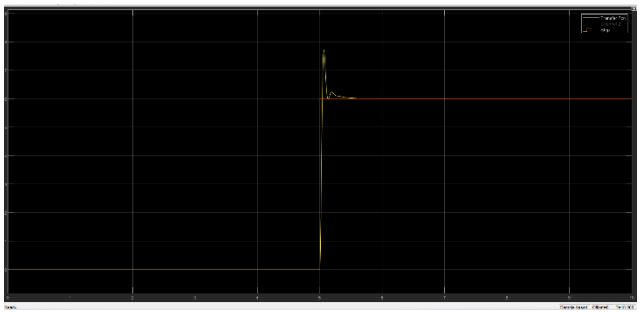


Imagen 24. *Respuesta del sistema en lazo cerrado ante una entrada escalón – simulación en Simulink.*

10. SEGUNDA FORMULACIÓN DE VALORES PID (CÁLCULO MANUAL)

Con el fin de **validar los resultados obtenidos por la herramienta PID Tuner** de Matlab y comparar el desempeño del sistema frente a otras configuraciones, se realizó un **cálculo manual alternativo de los parámetros PID** utilizando un enfoque clásico basado en el análisis de la respuesta del sistema.

Este procedimiento se fundamenta en la estimación de la ganancia proporcional (Kp) el tiempo de retardo (t_0) y la constante de tiempo del sistema (τ), a partir de los datos experimentales obtenidos del comportamiento del monocóptero ante una entrada escalón.

10.1 Cálculo de parámetros

1. Estimación de ganancia proporcional (Kp)

Se parte de una entrada escalón del 50% en la señal de control, lo cual activa el motor para iniciar el desplazamiento:

- Setpoint aplicado:

$$Spam = 50\ CM$$

$$\Delta M = 50 - 0 = 50\%$$

- Respuesta del sistema (variación de la variable de proceso):

$$\Delta PV = Valor\ tabla\ final - valor\ tabla\ inicial$$

$$\Delta PV = 3 - 67 = -64$$

- Cambio porcentual de salida:

$$\Delta C = -\frac{64}{50} * 100 = -128\%$$

- Ganancia proporcional estimada:

$$KP = \frac{-128\%}{50\%} = -2.56$$

(Signo negativo por tratarse de una respuesta descendente)

2. Estimación del tiempo de respuesta y retardo

Se utilizaron criterios clásicos para determinar dos puntos característicos de la curva de respuesta:

- **Cambio significativo (28.3% del total):**

$$(\Delta PV * 0.283) + valor\ tabla\ inicial$$

$$= (-64 * 0.283) + 67 = 48.888cm$$

- Estabilidad (63.2% del total):

$$(\Delta PV * 0.632) + valor\ tabla\ inicial$$

$$= (-64 * 0.632) + 67 = 26.52cm$$

Mediante **interpolación lineal** entre puntos medidos, se estimaron los tiempos correspondientes:

$X1 = 49\ cm$	$Y1 = 0.74\ seg$
$X = 48.888cm$	$Y1 = 0.7412\ seg$
$X2 = 48\ cm$	$Y1 = 0.75\ seg$

$$t1 = 0.7412\ seg$$

$X1 = 27\ cm$	$Y1 = 1.11\ seg$
$X = 26.552\ cm$	$Y1 = 1.11448\ seg$
$X2 = 26cm$	$Y1 = 1.12\ seg$

$$t2 = 1.11448\ seg$$

Hallamos τ

$$\tau = \frac{3}{2}(t2 - t1)$$

$$= \frac{3}{2}(1.11448 - 0.7412) = 0.55992$$

Para t_0

$$t_0 = t2 - \tau$$

$$= 1.11448 - 0.55992 = 0.55456$$

10.2 CÁLCULO DE PARÁMETROS PID

A partir de los valores estimados:

$$KP = 2.56$$

$$Ki = \frac{KP}{\tau} = \frac{2.56}{0.55992} = 4.572$$

$$Kd = Kp * t_0 = 2.56 * 0.55456 = 1.4196$$

Obteniendo así los valores de PID natural.
ahora comparando tenemos una varicacion de los datos,

PID TUNER	***CALCULADO***
$Kp = 2.001$	$Kp = 2.56$
$Ki = 0.449$	$Ki = 4.572$
$Kd = 0.413$	$Kd = 1.4196$

Se observa una **coincidencia cercana en el valor de** Kp , mientras que Ki y Kd difieren notablemente. Esta diferencia se debe principalmente a que:

- El método manual simplifica el comportamiento del sistema a un modelo de primer orden con retardo (FOPDT), sin tener en cuenta componentes de mayor orden, ruido o no linealidades.
- PID Tuner realiza una estimación optimizada basada en ajuste de modelos dinámicos reales obtenidos a partir de los datos, utilizando métodos numéricos avanzados para minimizar el error de predicción y maximizar el ajuste.

9. EVALUACIÓN PRÁCTICA DEL SISTEMA CON AMBOS PID

9.1 PID AJUSTADO CON MATLAB

Al implementar el sistema utilizando los parámetros obtenidos con **PID Tuner**, se observó:

- Estabilización rápida y sin oscilaciones significativas.
- Buena capacidad de rechazo ante perturbaciones, logrando volver al equilibrio con rapidez.
- Excelente desempeño en el seguimiento de referencias.

9.2 PID calculado manualmente

Con el controlador PID basado en los valores calculados manualmente, se obtuvo:

- Una respuesta estable y eficaz.
- Menor sobreimpulso en algunos casos, pero con una respuesta ligeramente más lenta ante perturbaciones.
- Mayor sensibilidad a errores acumulativos debido a la alta ganancia integral Ki.

Conclusión de la comparación

Ambos métodos ofrecen resultados funcionales y útiles para el control del monocóptero. No obstante, el ajuste automático mediante **PID Tuner proporciona una solución más balanceada**, especialmente en sistemas reales donde intervienen factores como ruido, fricción y no linealidades que no se contemplan fácilmente en métodos manuales.

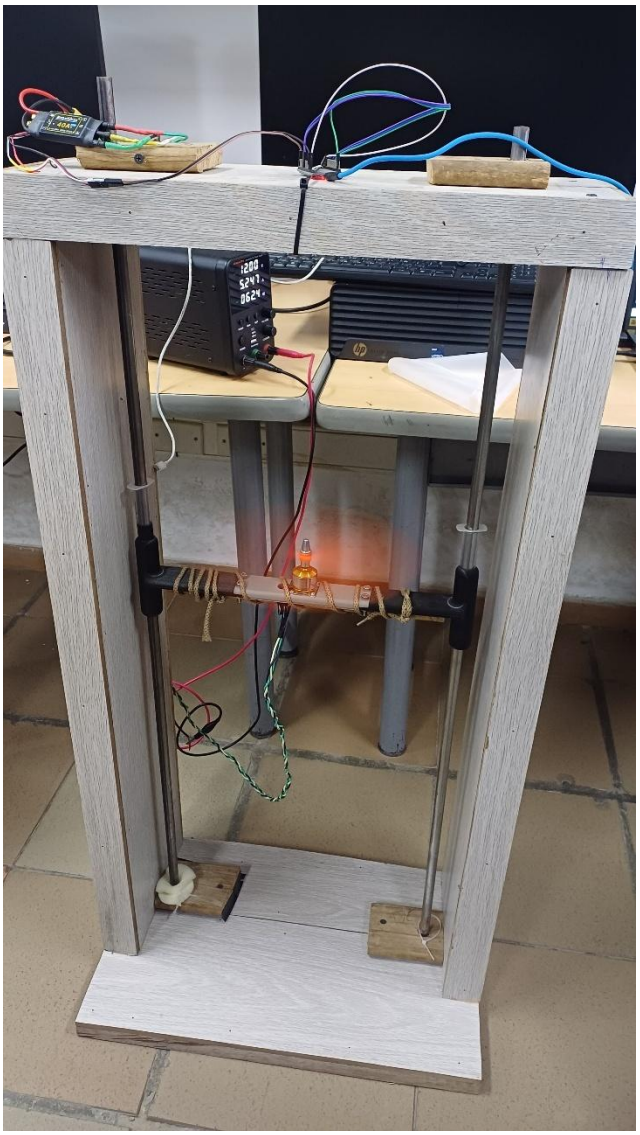


Imagen 25. *Resultado final.*

11. **Control Difuso**
    7.1. Definición de universos del discurso (entradas y salida)
    7.2. Diseño de funciones de membresía
    7.3. Formulación de la matriz de inferencia (FAM) con justificación de reglas
12. **Control por Redes Neuronales**
    8.1. Preparación del conjunto de datos (dataset)
    8.2. Arquitectura de la red (capas, neuronas, activaciones)
    8.3. Algoritmo de entrenamiento y parámetros (optimizador, tasa de aprendizaje)
    8.4. Evaluación y comparación de resultados
13. **Comparación Global**
    9.1. Resultados cuantitativos y cualitativos de cada método
    9.2. Ventajas y limitaciones de cada controlador
14. **Conclusiones**
    10.1. Principales hallazgos
    10.2. Recomendaciones y posibles extensiones